

The Case of the Slow Miss

A StableHashMap performance detective story — with math, instrumentation, and benchmarks.

Prologue: the one operation that should be boring

A hash map miss is supposed to be the easy path: no value to return, no insertion to perform, no rehashing — ideally just a small amount of metadata and an early stop at an empty slot.

So when our reference-stable hash map (“StableHashMap”: pointers/references stay valid across insert/reserve) started losing badly on Find(miss) at large sizes, it wasn’t just disappointing — it was suspicious.

Because if a miss is slow, it usually means you’re doing work you didn’t intend to do.

This is the story of how we hunted that work down, using a benchmark harness designed to resist “benchmark lies,” a purpose-built Miss Diagnostics counter, a little probability, and — in the end — a one-line bitmask that changed everything.

Act I: the smoke test that started it all

The benchmark suite is intentionally strict:

- Round-robin execution: each measured run executes exactly one timed iteration per library; library order is randomized per run.
- CPU frequency stability: the harness waits until frequency variance is below a threshold before timing (critical on Windows power management).
- Medians are the primary statistic.

In that harness, at $N = 1,000,000$, the original build produced a miss result that looked like this:

Map	Find(miss) ns/op (Core)
StableHashMap + SplitMix64	17.61
StableHashMap[Block] + SM64	13.16
boost::unordered_node_map + SM64	5.52

A reference-stable map is allowed to be slower than a flat map. But on misses, why would it be ~3× slower than boost’s node map? That question became the investigation.

Act II: first suspects — hash quality and tag correlation

Before we assumed “the table logic is wrong,” we started with two common performance killers in open addressing.

Bad hashes

On many platforms, `std::hash<int>` behaves like identity for integers. That can create patterns (especially in the low bits) that are invisible at small N and painful at large N.

So `FastHashMap` and `StableHashMap` gained a built-in `SplitMix64` finalizer by default, with an opt-out marker (`is_avalanching`) for already-good hashes. That’s defensive engineering, not a micro-optimization.

Using the wrong bits for H2

`StableHashMap` uses a small H2 tag (7 bits) stored in the control byte array. SIMD compares H2 across the group to quickly find “maybe” candidates. But the bucket index is derived from low bits via `hash & mask`.

If H2 also uses low bits, you risk correlation between “where the key lands” and “which tags look similar,” inflating tag matches and therefore node dereferences on misses. So Option A moved H2 extraction to the high bits of the hash, decoupling H2 from the bucket index.

The “don’t guess” check: H2-biased miss sets

To test H2 sensitivity without contrived keys, `MissDiag` generates two additional miss sets by biasing `SplitMix64` bits:

- H2-biased (low7)
- H2-biased (high7)

If one of those increases `Tag/miss` and `Eq/miss`, you’ve found a false-positive sensitivity. Any fix had to hold for random misses and for biased misses.

Act III: build a microscope, not a bigger hammer

Even after the “hash/H2 hygiene” improvements, we still saw misses that didn’t feel like they were terminating early. At this point there were too many plausible suspects — probe behavior, false positives, node indirections, or a subtle miss-path bug. Guessing is expensive, so instead we built a microscope.

MissDiag: turning a miss into a structured measurement

Miss Diagnostics isolates misses and reports:

- `ns/miss`: latency per miss
- `Eq/miss`: `key_equal` calls per miss (candidate compares)
- `Hash/miss`: hash computations per miss (should be ~1)

- Grp/miss: probe groups visited per miss
- FullSlots/miss: occupied slots scanned across visited groups
- FullGrp/miss: groups with no empties encountered
- Tag/miss: tag matches per miss

This instrument does two jobs: it tells us where the time goes, and it tells us whether the “miss should terminate early” expectation holds.

Act IV: the numbers that didn’t make sense (until they did)

Here is the critical before/after comparison at N = 1,000,000, reserve = 1,000,000, random misses:

Scenario	Median(ns)	Eq/miss	Tag/miss	Grp/miss	FullSlots/m
Before	10.54	0.12	0.12	1.00	15.27
Option B					
After Option B	3.33	0.01	0.01	1.00	15.27

Nothing about table occupancy changed (FullSlots/m stayed 15.27). But candidate work collapsed by ~12× and latency by ~3×. That Eq/miss collapse was the tell.

A quick piece of math to interpret those counters

StableHashMap’s tag is 7 bits, so with a well-mixed hash $P(\text{tag match}) \approx 1/128$ per occupied slot. Expected tag matches scale with how many occupied slots you actually consider.

In the “before” result: FullSlots/miss ≈ 15.27 , and $15.27/128 \approx 0.119$ — and Tag/miss was 0.12. That’s too perfect. It suggests we were effectively processing tag matches across all occupied slots in the group window, even on misses that should terminate early.

Act V: the real culprit — ordering

Here’s the key structure in the original find_slot() loop (simplified):

```
match_mask = match(h2);    // candidates across the whole group

while (match_mask) {      // compare each candidate
    deref node;
    key_equal(...);
    clear lowest bit;
}

if (match_empty()) return npos; // only now do we learn the search should stop
```

Notice the order: compute tag matches across all positions, process every match (which can dereference nodes), then check whether an empty existed.

But on a miss, once there's an empty in the group window, the search terminates — and any occupied slots after the first empty cannot contain the key for this probe window. So tag matches after the first empty are guaranteed false positives, and each false positive is a node dereference plus a key compare.

Act VI: Option B — the one-mask fix

Option B implements a simple rule: stop processing tag matches after the first empty in the group.

Mechanically:

- Compute `match_mask` for H2.
- Compute `empty_mask`.
- Find the first empty bit in the group.
- Mask off any tag matches after that bit.

Pseudocode:

```
empty_mask = match_empty();
if (empty_mask) {
    first_empty_bit = lowest_set_bit(empty_mask);
    match_mask &= (first_empty_bit - 1); // keep only bits before the first empty
}
```

Now the algorithm only performs candidate comparisons in the prefix that is actually reachable for this probe.

Act VII: validation — the “this can’t be a fluke” checklist

A good optimization isn’t “faster once.” It’s faster across sizes, stable across CPU states, and consistent with counters.

After Option B:

- Core Find(miss) at N = 1,000,000 dropped dramatically:
- • StableHashMap + SplitMix64: 17.61 → 3.33 ns
- • StableHashMap[Block] + SM64: 13.16 → 3.06 ns
- MissDiag Eq/miss dropped from 0.12 → 0.01 at N = 1,000,000.

And the math now lines up with the expected work. At $N = 1,000,000$ reserve = 1,000,000, capacity rounds up to a power of two ($\approx 2,097,152$), so the actual load factor is $LF \approx 1,000,000 / 2,097,152 \approx 0.477$ ($P(\text{empty}) \approx 0.523$).

If empties are common, the expected number of full slots before the first empty is roughly $LF/(1-LF) \approx 0.91$, so expected tag matches before the first empty is $0.91/128 \approx 0.007$ — exactly the order of magnitude we observed ($\text{Tag}/\text{miss} \approx 0.01$).

Act VIII: the supporting cast — harness design and the slim bench

Once MissDiag existed, we didn't need to run the entire suite for every hypothesis. So we slimmed the benchmark down to the three loops that mattered for this investigation:

- Core operations
- Pathological erase churn
- Slim MissDiag

That makes “hypothesis → patch → verify” cycles much faster, without losing rigor.

Epilogue: what this teaches (beyond hash maps)

- Start with a symptom that is hard to dismiss (“miss is slow”).
- Build instrumentation that turns hunches into measurements (MissDiag counters).
- Use math as a sanity check, not a substitute for data.
- Change one thing at a time, and demand that counters move in the expected direction.
- Bake the lesson into documents so the next person doesn't have to rediscover it.

If Chasing Speed is the mindset, and Design History is the map of past decisions, then this case study is the moment where measurement, math, and code all agreed on the same truth:

The empties were there all along. We just weren't letting them end the search soon enough.

Where this fits in the Teaching Pack

This narrative is intended as the “front door” document. For deeper detail, cross-reference:

- Chasing_Speed.md — measurement-driven iteration mindset
- FastHashMap_Design_History.md — design evolution and tradeoffs
- CaseStudy_MissDiag_FirstEmptyMask.md — technical deep dive and implementation notes